



Assignment No: 03

Marks: 80

Section: BCS (A to J)

CLO: 03

Deadline: April 02, 2026 before 02:00pm

Semester: Spring-26

Instructions:

1. Plagiarized work will result in zero marks.
 2. No retake or late submission will be accepted.
 3. Attach complete code, results, and screenshot for questions that require programming solutions. Programs/codes should be typed.
 4. **Complete assignment is to be submitted in soft-copy as well as in hard-copy. Submit the hardcopy before the deadline through CR, and softcopy on GCR.**
 5. The softcopy should be a single PDF file of your complete assignment including programming and non-programming questions.
 6. The PDF file should be according to the following format: id_section_A1 e.g., i25-123456_A_A2. A2 in the end denotes Assignment 2. **The title page must include complete student information, including name, section, id, course name, and assignment number.**
 7. The images of a by-hand solution should be properly scanned. You can use any mobile application such as CamScanner or Adobe Scan for scanning. Each of these applications allows you to export pdf or image files which you can use to combine with your programming solutions. Do not attach direct images from the camera application of your mobile phone, or screenshots.
 8. Python is the only approved programming language.
-

Part 1: Resource Optimization

Problem 1: Resource Optimization in an AI Cloud Service

A software engineering team is deploying a cloud-based AI inference system that processes real-time user requests. The performance of the system depends on two computational resources:

x : number of CPU processing units allocated to the system

y : number of GPU acceleration units allocated to the system

Due to nonlinear interactions between CPU preprocessing and GPU parallel computation, the overall throughput efficiency of the system (measured in thousands of requests per second) is modeled by

$$T(x, y) = 5x^2 + 3y^2 + 4xy$$

The cloud infrastructure team imposes a total hardware allocation constraint because of budget and energy consumption considerations. The weighted resource usage must satisfy

$$2x + y = 12$$

where CPU units count twice as much as GPU units in the cost model.

In addition to this budget constraint, the system must also satisfy engineering limitations:

The number of CPU and GPU units cannot be negative

$$x \geq 0, \quad y \geq 0$$

Due to thermal limits of the GPU cluster, the GPU allocation cannot exceed 5 units

$$y \leq 5$$

Your Task

1. Formulate the above problem as a nonlinear programming problem including all equality and inequality constraints.
2. Solve the problem using the Lagrange multiplier method considering the equality constraint.
3. Apply the Karush–Kuhn–Tucker (KKT) conditions to incorporate the inequality constraints and determine whether any inequality constraint becomes active at the optimal point.
4. Compute the Hessian matrix of the objective function and use second-order conditions to determine the nature of the optimal solution.

Python

1. Using a numerical solver, implement the optimization problem with equality and inequality constraints.
2. Compare the numerical solution with the analytical solution obtained from Lagrange multipliers and KKT condition.

Problem 2: Optimizing a Polynomial Function for Real-Time Software Prediction

A software engineering team is designing a lightweight real-time scoring system for an online service. The system predicts a performance metric y based on two input features x_1 and x_2 , which are numeric metrics collected from user interactions.

The team decides to model the output using a simple polynomial function of the form:

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_2^2 + w_5x_1x_2$$

Here, $w = [w_1, w_2, w_3, w_4, w_5]^T$ are the parameters to optimize, and the bias term is ignored for simplicity.

The prediction error is measured using the mean squared error (MSE) loss function:

$$L(w) = \frac{1}{2}(\hat{y} - y)^2$$

Your objectives are to:

1. Represent the forward computation of $\hat{y}$ using a computational graph.
2. Apply automatic differentiation to compute $\nabla_w L(w)$.
3. Perform a gradient descent update on the weights.
4. Discuss the selection of learning rate λ using a Lipschitz condition for a convex quadratic approximation of the loss function.

Python

1. Compute $\hat{y}$ in Python given x_1, x_2 and w .
2. Use automatic differentiation to compute $\nabla_w L(w)$ in Python.
3. Perform one gradient descent update: $w \leftarrow w - \lambda \nabla_w L(w)$.

Given numerical example for clarity:

- Input features: $x_1 = 1, x_2 = 2$
- Target output: $y = 10$
- Initial weights: $w_1 = 0.5, w_2 = 0.5, w_3 = 0.1, w_4 = 0.1, w_5 = 0.2$

Problem 3: Optimizing Cloud Resource Allocation with Nonlinear Convex Cost

A software engineering team is deploying a cloud-based service that dynamically allocates virtual machines (VMs) to incoming user requests. Each VM has a processing capacity x (in arbitrary units), and the total system cost is a combination of quadratic energy consumption and nonlinear penalty for under-provisioning:

$$C(x) = \alpha x^2 + \beta e^{-kx}$$

where:

- $x \geq 0$ is the number of allocated VM units,
- $\alpha > 0$ represents the energy cost per VM squared (quadratic energy term),
- $\beta > 0$ and $k > 0$ control the nonlinear penalty for insufficient capacity,
- e^{-kx} ensures the penalty decreases as more resources are allocated.

The goal of the team is to find the optimal x^* that minimizes the cost.

Objectives for students:

- Identify that $C(x)$ is nonlinear but convex for reasonable parameter choices ($\alpha, \beta, k > 0$).
- Compute the gradient $\frac{dC}{dx}$ using differentiation.
- Apply gradient descent to iteratively update x toward x^* .
- Discuss learning rate selection, ensuring convergence even for a nonlinear convex function.
- Use backtracking line search or Lipschitz approximation for step-size selection (Just the method and further criteria, no need to solve).

Numerical example for clarity:

- $\alpha = 1, \beta = 5, k = 0.5$
- Initial allocation: $x_0 = 1$

Part 2: Taylor's Formula (14.7)

Problem 4:

A software engineering team is developing a real-time graphics engine that dynamically adjusts rendering quality based on two factors: frame processing delay x (in milliseconds) and GPU temperature rise y (in °C). To predict how small changes in these two variables affect overall performance, the team models the performance score $f(x, y)$ of the engine as:

$$f(x, y) = e^{-0.1x} \cos(0.5y) + xy^2$$

Here, the term $e^{-0.1x}$ represents the performance drop due to additional frame delay, $\cos(0.5y)$ captures oscillatory performance changes due to GPU temperature fluctuations, and the interaction term xy^2 models the combined effect of delay and temperature on performance.

The team wants to analyze the quadratic behavior of $f(x, y)$ near the optimal operating point $(0, 0)$. Using the second-order Taylor expansion, approximate $f(x, y)$ near the origin to understand how small variations in frame delay and GPU temperature jointly affect performance.